

面试手撕题集合（LLM/Infra/算法方向）

2025-12-25 · marshall

经典Leetcode题目

反转链表

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */
class Solution {
public:
    ListNode* reverseList(ListNode* head) {
        ListNode* cur = head;
        ListNode* pre = nullptr;
        ListNode* tmp = new ListNode();
        while(cur){
            tmp = cur.next
            cur.next = pre
            pre = cur
            cur = tmp
        }
        return pre;
    }
};
```

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def reverseList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        cur = head
        pre = None
        tmp = ListNode()

        while cur != None:
            tmp = cur.next
            cur.next = pre
            pre = cur
            cur = tmp
```

```
return pre
```

链表重排

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */
class Solution {
public:
    ListNode* reverseList(ListNode* head){
        ListNode* cur = head;
        ListNode* pre = nullptr;
        ListNode* tmp = new ListNode();

        while(cur){
            tmp = cur->next;
            cur->next = pre;
            pre = cur;
            cur = tmp;
        }
        return pre;
    }

    ListNode* findMid(ListNode* head){
        ListNode* fast = head;
        ListNode* slow = head;
        while(fast->next != nullptr && fast->next->next != nullptr){
            slow = slow->next;
            fast = fast->next->next;
        }
        return slow;
    }

    void mergeList(ListNode* l1, ListNode* l2){
        ListNode* l1_tmp;
        ListNode* l2_tmp;

        while(l1 != nullptr && l2 != nullptr){
            l1_tmp = l1->next;
            l2_tmp = l2->next;

            l1->next = l2;
            l1 = l1_tmp;

            l2->next = l1;
            l2 = l2_tmp;
        }
    }
}
```

```

void reorderList(ListNode* head) {
    ListNode* mid = findMid(head);
    ListNode* l1 = head;
    ListNode* l2 = mid->next;
    mid->next = nullptr;
    l2 = reverseList(l2);
    mergeList(l1, l2);
}
};

```

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def reverseList(self, head):
        cur = head
        pre = None
        tmp = ListNode()

        while cur != None:
            tmp = cur.next
            cur.next = pre
            pre = cur
            cur = tmp
        return pre

    def findMid(self, head):
        slow, fast = head, head
        while (fast.next != None and fast.next.next != None):
            slow = slow.next
            fast = fast.next.next
        return slow

    def mergeList(self, l1, l2):
        l1_tmp, l2_tmp = ListNode(), ListNode()
        while (l1 != None and l2 != None):
            l1_tmp = l1.next
            l2_tmp = l2.next

            l1.next = l2
            l1 = l1_tmp

            l2.next = l1
            l2 = l2_tmp

    def reorderList(self, head: Optional[ListNode]) -> None:
        """
        Do not return anything, modify head in-place instead.
        """
        mid = self.findMid(head)
        l1 = head
        l2 = mid.next
        mid.next = None
        l2 = self.reverseList(l2)

```

```
self.mergeList(l1, l2)
```

中序遍历二叉树(递归)

```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
 * };
 */
class Solution {
public:
    void inorder(TreeNode* root, vector<int>& res){
        if (!root) return;
        inorder(root->left, res);
        res.push_back(root->val);
        inorder(root->right, res);
    }
    vector<int> inorderTraversal(TreeNode* root) {
        vector<int> res;
        inorder(root, res);
        return res;
    }
};
```

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def inorder(self, root, res):
        if not root:
            return
        self.inorder(root.left, res)
        res.append(root.val)
        self.inorder(root.right, res)
    def inorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
        res = []
        self.inorder(root, res)
        return res
```

中序遍历二叉树(非递归)

```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
 * };
 */
class Solution {
public:
    vector<int> inorderTraversal(TreeNode* root) {
        vector<int> res;
        stack<TreeNode*> st;
        while(root != nullptr || !st.empty()){
            while(root != nullptr){
                st.push(root);
                root = root->left;
            }
            root = st.top();
            st.pop();
            res.push_back(root->val);
            root = root->right;
        }
        return res;
    }
};
```

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def inorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
        st = []
        res = []
        while root or st:
            while root:
                st.append(root)
                root = root.left
            root = st.pop()
            res.append(root.val)
            root = root.right
        return res
```

最长连续序列

```
class Solution {
public:
    int longestConsecutive(vector<int>& nums) {
        unordered_set<int> num_set;
        for (int num : nums) num_set.insert(num);

        int max_len = 0;

        for (int num : num_set){
            if (!num_set.count(num - 1)){
                int cur_num = num;
                int cur_len = 1;
                while (num_set.count(cur_num+1)){
                    cur_num += 1;
                    cur_len += 1;
                }
                max_len = max(max_len, cur_len);
            }
        }
        return max_len;
    }
};
```

```
class Solution:
    def longestConsecutive(self, nums: List[int]) -> int:
        max_len = 0
        num_set = set(nums)

        for num in num_set:
            if num - 1 not in num_set:
                cur_num = num
                cur_len = 1
                while cur_num + 1 in num_set:
                    cur_num += 1
                    cur_len += 1
                max_len = max(max_len, cur_len)
        return max_len
```

蚂蚁爬杆

```
def calculate_ant_time(L, positions):
    if len(positions) == 0:
        return (0.0, 0.0)
    min_time = []
    max_time = []
    for pos in positions:
        min_time.append(min(pos, L-pos))
        max_time.append(max(pos, L-pos))
    return max(min_time), max(max_time)
```

二叉树层序遍历

```
def levelOrder(root):
    if not root:
        return
    res = []
    st = [root]

    while st:
        lay = []
        lay_val = []
        for node in st:
            lay_val.append(node.val)
            if node.left:
                lay.append(node.left)
            if node.right:
                lay.append(node.right)
        st = lay
        res.append(lay_val)
    return res

def levelOrder(root):
    if not root:
        return
    res = []
    st = [root]

    while st:
        lay = []
        lay_val = []
        for node in st:
            lay_val.append(node.val)
            if node.left:
                lay.append(node.left)
            if node.right:
                lay.append(node.right)
        st = lay
        res.append(lay_val)
    return res
```

Triton/CUDA

Base matrix multiply

```
__global__ void cudaMatrixMultiplyBase(float *A, float *B, float *C, int M, int N, int K){
    int row = blockIdx.x * blockDim.x + threadIdx.x;
    int col = blockIdx.y * blockDim.y + threadIdx.y;

    if (row < M && col < N){
        float sum = 0.0f;
        for (int k = 0; k < K; ++k){
            sum += A[row*K + k] * B[k*N + col]; // row行k列, k行col列
        }
        C[row*N + col] = sum;
    }
}
```

```

    }
    C[row * N + col] = sum; // row行col列
}
}

```

Dropout

```

import triton
import triton.language as tl

@triton.jit
def _seeded_dropout(
    x_ptr,
    output_ptr,
    n_elements,
    p,
    seed,
    BLOCK_SIZE: tl.constexpr,
):
    # compute memory offsets of elements
    pid = tl.program_id(axis=0)
    block_start = pid * BLOCK_SIZE
    offsets = block_start + tl.arange(0, BLOCK_SIZE)
    # load data from x
    mask = offsets < n_elements
    x = tl.load(x_ptr + offsets, mask=mask)
    # randomly prune it
    random = tl.rand(seed, offsets)
    x_keep = random > p
    # write-back
    output = tl.where(x_keep, x / (1-p), 0,0)
    tl.store(output_ptr + offsets, output, mask=mask)

```

layer norm

大矩阵规约求和

online softmax

其他

简易prefix caching

```

"""
Prefix-based matching

```

```

Key: tokens
Value: KV cache tensors
Token 1: ABCDE -> [KV1, KV2, KV3, KV4, KV5]
Token 2: ABCDF -> [KV1, KV2, KV3, KV4, KV6]

kv_cache_store.store("ABCDE", [KV1, KV2, KV3, KV4, KV5])
kv_cache_store.retrieve("ABCDF") -> [KV1, KV2, KV3, KV4]
"""

class KVCacheStore:
    def __init__(self):
        self.chunk_store = {}
        self.prefix_index = {}

    def store

```

MHA

```

from einops import einsum, rearrange
import torch.nn as nn

class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, n_heads):
        self.n_heads = n_heads
        self.head_dim = d_model // n_heads

        self.w_qkv = nn.Linear(d_model, 3 * d_model)
        self.output_proj = nn.Linear(d_model, d_model)

        self.rope = RoPE()

    def forward(self, x, mask=None):
        seq_len = x.shape[-2]
        qkv = self.w_qkv(x)
        q, k, v = rearrange(qkv, "... seq (three head head_dim) -> three ... head seq head_dim",
                             three=3, head=self.num_heads, head_dim = self.head_dim)

        token_positions = torch.arange(seq_len)
        q = self.rope(q, token_positions=token_positions)
        k = self.rope(k, token_positions=token_positions)

        scores = einsum(q, k, 'batch ... q head_dim, batch ... k head_dim -> batch ... q k') /
        math.sqrt(self.head_dim)
        if mask is not None:
            scores = scores.masked_fill(~mask, float('-inf'))
        attn_weights = F.softmax(scores, dim=-1)
        attn = einsum(attn_weights, v, 'batch ... q k, batch ... k head_dim -> batch ... q
        head_dim')
        attn = rearrange(attn, "... head seq head_dim -> ... seq (head head_dim)",
                         head=self.num_heads, head_dim=self.head_dim)
        output = self.output_proj(attn)

        return output

```

torch实现qwen3架构

RoPE*

LayerNorm

```
import torch
import torch.nn as nn

class RMSNorm(nn.Module):
    def __init__(self, dim, eps=1e-6):
        super().__init__()
        self.eps = eps
        self.weight = nn.Parameter(torch.ones(dim))

    def forward(self):
        rms = torch.sqrt(torch.mean(x**2, dim=-1, keepdim=True) + self.eps)
        return x / rms * weight

class RMSNorm(nn.Module):
    def __init__(self, dim, eps=1e-6):
        super().__init__()
        self.eps = eps
        self.weight = nn.Parameter(torch.ones(dim))

    def forward(self):
        rms = torch.sqrt(torch.mean(x**2, dim=-1, keepdim=True) + self.eps)
        return x / rms * weight

class LayerNorm(nn.Module):
    def __init__(self, hidden_size, eps=1e-6):
        super(LayerNorm, self).__init__()
        self.gamma = nn.Parameter(torch.ones(hidden_size))
        self.beta = nn.Parameter(torch.zeros(hidden_size))
        self.eps = eps

    def forward(self, x):
        mean = x.mean(dim=-1, keepdim=True)
        var = x.var(dim=-1, keepdim=True)
        x_norm = (x - mean) / (torch.sqrt(var+self.eps))
        output = self.gamma*x_norm + self.beta
        return output

    def forward(self, x):
        x_mean = x.mean(dim=-1, keepdim=True)
        x_var = x.var(dim=-1, keepdim=True)
        x_norm = (x - x_mean) / (torch.sqrt(x_var + self.eps))
        output = self.gamma*x_norm + self.beta
        return output
```

Softmax

```
def softmax(x):  
    max_x = max(x)  
    exp_x = [math.exp(i - max_x) for i in x]  
    sum_exp_x = sum(exp_x)  
    output = [ e / sum_exp_x for e in exp_x]  
    return output
```